

WSI Viewer Release Cheat Sheet

Development -> Staging -> Production rehearsal -> Production -> Tag

Environments

Environment	Port	Runtime / identity	Data boundary
Development	8081	Live source; red banner	Deidentified development images and annotations
Staging	8082	Candidate JAR; yellow banner	Deidentified staging images and annotations
Rehearsal	8083	Exact staging JAR; production mode; loopback only	Deidentified production-marked images; rehearsal annotations
Production	8080	Frozen validated JAR; no banner	Authorized clinical images and production annotations

Normal release

One-command monitored workflow

```
./ops/wsi-release cycle --tag NAME
./ops/wsi-release cycle --dry-run
./ops/wsi-release cycle --resume
```

Normal: cycle --tag NAME runs safe phases in order. Diagnostic --step commands are listed on page 2.

Automated phases

1. Repository/environment/production preflight
2. Development tests, restart and verification
3. Publish tested feature commit
4. Build/install/verify staging
5. Copy exact JAR to isolated rehearsal
6. Final identity, health and backup-space preflight
7. Verified backup then production promotion
8. Production QC and optional tag

Exact human gates

```
Development browser QC: y/n
Staging browser QC: y/n
Rehearsal browser QC: y/n
Promotion: y/n
Production browser QC: y/n
Publish saved tag: y/n
```

Browser success is never inferred. Production is not stopped until its complete backup verifies. Failure prints exact rollback guidance; rollback is never automatic.

State and detailed log

```
.runtime/run/release-cycle.state
.runtime/log/cycle-<id>.log
```

Every browser gate: banner/title, login, image discovery/open/switch, tiles/pan/zoom, channel/display controls, full annotation naming/edit/delete/persistence and global Show/Hide, exports, isolated annotations, clean console/network, and no sustained delay.

Troubleshooting and Recovery

Status and verification

```
wsi production status
wsi staging status
wsi rehearsal status
wsi development status
./ops/wsi-release status
./ops/wsi-release verify staging
./ops/wsi-release verify rehearsal
./ops/wsi-release verify production
```

Logs

```
wsi production logs
wsi staging logs
wsi rehearsal logs
wsi development logs
```

Press **Control-C** to stop following a log.

Rollback

```
./ops/wsi-release history
./ops/wsi-release rollback --step
./ops/wsi-release verify production
```

Rollback restores the prior JAR, metadata, checksum and configuration. It **never restores or overwrites annotations**.

Troubleshooting modes

```
./ops/wsi-release cycle --step
./ops/wsi-release cycle --resume --step
./ops/wsi-release cycle --resume --verbose
./ops/wsi-release stage --dry-run
./ops/wsi-release stage --step --verbose
./ops/wsi-release rehearse --dry-run
./ops/wsi-release promote --dry-run
./ops/wsi-release history
```

--dry-run: plan only. **--step:** Enter runs, p repeats, q exits safely. Use `./ops/wsi-release` if the installed command is missing. Never bypass PROMOTE or ROLLBACK.

Stop conditions

- Dirty Git tree or failed required test
- Git HEAD differs from staging commit
- Recorded/calculated SHA-256 differ
- Staging and rehearsal differ
- Environment identity, marker, root or port mismatch
- Production backup validation fails
- New 403, 500, unhandled JavaScript error, lost annotation, blank image or routine delay

Do not repair a mismatch by editing build metadata or checksum files.

Marker rule

Identity	Required marker
development	.wsi-environment-development
staging	.wsi-environment-staging
production / rehearsal	.wsi-environment-production

Timing: Image-switch total ends at OpenSeadragon open and may not include all visible tiles. Use browser Network timing and server logs for repeatable delays.

Privacy: Never copy production slides into development, staging or rehearsal. Verify filenames, metadata, embedded labels, thumbnails, macro images and associated files. Never synthesize a missing embedded label/thumbnail from diagnostic pixels.

Exactly one environment marker must exist. A missing, multiple or cross-environment marker prevents startup before the server accepts requests.

Manual WSI ingestion

Scanner-independent manual readiness + repeated whole-tree observations + atomic promotion

Manual readiness gate: Confirm the scanner or copy appears finished before SEAL. Filesystem stability reduces risk but cannot prove acquisition completeness.

1. Start and prepare

```
cd /Users/dm026/Downloads/wsi-server_works
source ops/wsi-ingest.conf
./ops/wsi-ingest status
```

Place one complete top-level directory beneath **/Users/dm026/wsi-ingest-staging**. The directory is one indivisible batch containing every .vsi, companion data directory and associated file.

Never move a single .vsi without its companion data. Never reuse a promoted name; use unique names such as 2026-08-05_batch-01 and 2026-08-05_batch-02.

2. Seal and observe

```
./ops/wsi-ingest inspect DATASET
./ops/wsi-ingest seal DATASET # type SEAL
```

Do not modify the directory after sealing. The seal is observation 1. With validated defaults:

1. Wait at least 60 seconds.
2. Run **./ops/wsi-ingest observe DATASET**.
3. Wait at least another 60 seconds.
4. Run **./ops/wsi-ingest observe DATASET**.
5. Run **./ops/wsi-ingest history**; confirm three observations.

Never manually copy/merge the sealed batch into production, use --force, or bypass stability. Full safety, atomicity and recovery design: docs/WSI-INGESTION.md

3. Preflight and promote

```
./ops/wsi-ingest promote --dry-run DATASET
./ops/wsi-ingest promote --step DATASET
# type exactly PROMOTE
./ops/wsi-ingest history
./ops/wsi-ingest status
```

Confirm transaction ID, file count, byte count and absent production destination before --step.

Expected completion:
verified observations 3
sealed_pending_transactions: 0

Allow production live discovery to complete or use **Refresh images**. Confirm every expected image appears with expected channels.

Stop conditions

- Scanner/copy activity may continue
- Companion or associated file may be missing
- Dataset changed after seal
- Destination already exists
- File or byte count differs
- Roots missing, nested or on different filesystems
- Wrong environment markers
- Lock held or transaction ambiguous
- Any command reports FAIL